21、spi-nand驱动，升级到5.2 kernel

全志V3S嵌入式驱动开发（spi-nand驱动，升级到5.2 kernel）

原创

嵌入式-老费 已于 2024-10-09 10:51:51 修改

阅读量2.1k Open: 196806fd3bdd4f04e1a92a7697cf0354_MD5.png

收藏 1

Open: 95eb4f436541fdf1b2e1af49844ad0c3_MD5.png

点赞数

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发

Open: 135c7190ea3ccdc82fa1f0c0a41d08f0_MD5.png

基于全志soc的linux嵌入式开发专栏收录该内容](https://blog.csdn.net/feixiaoxing/category_12270221.html "基于全志soc的linux嵌入式开发")

73 篇文章¥49.90¥99.00

已订阅8折续费

[

Open: d9b05f322fadabcd81dc6c5654189e59_MD5.png

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益Open: ddf4d655e9aa3806864ef10c7c4a060b_MD5.png

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

Open: 1e4395309f4514fed3e1f5e89b2a25e6_MD5.png

摘要 本文介绍了全志V3S芯片如何进行SPI NAND闪存驱动开发，包括原理图解析、设备树配置、驱动代码编写及烧录验证过程。通过对MX35LF1GE4AB芯片的SPI接口分析，实现SPI NAND作为启动和存储系统数据的媒介。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

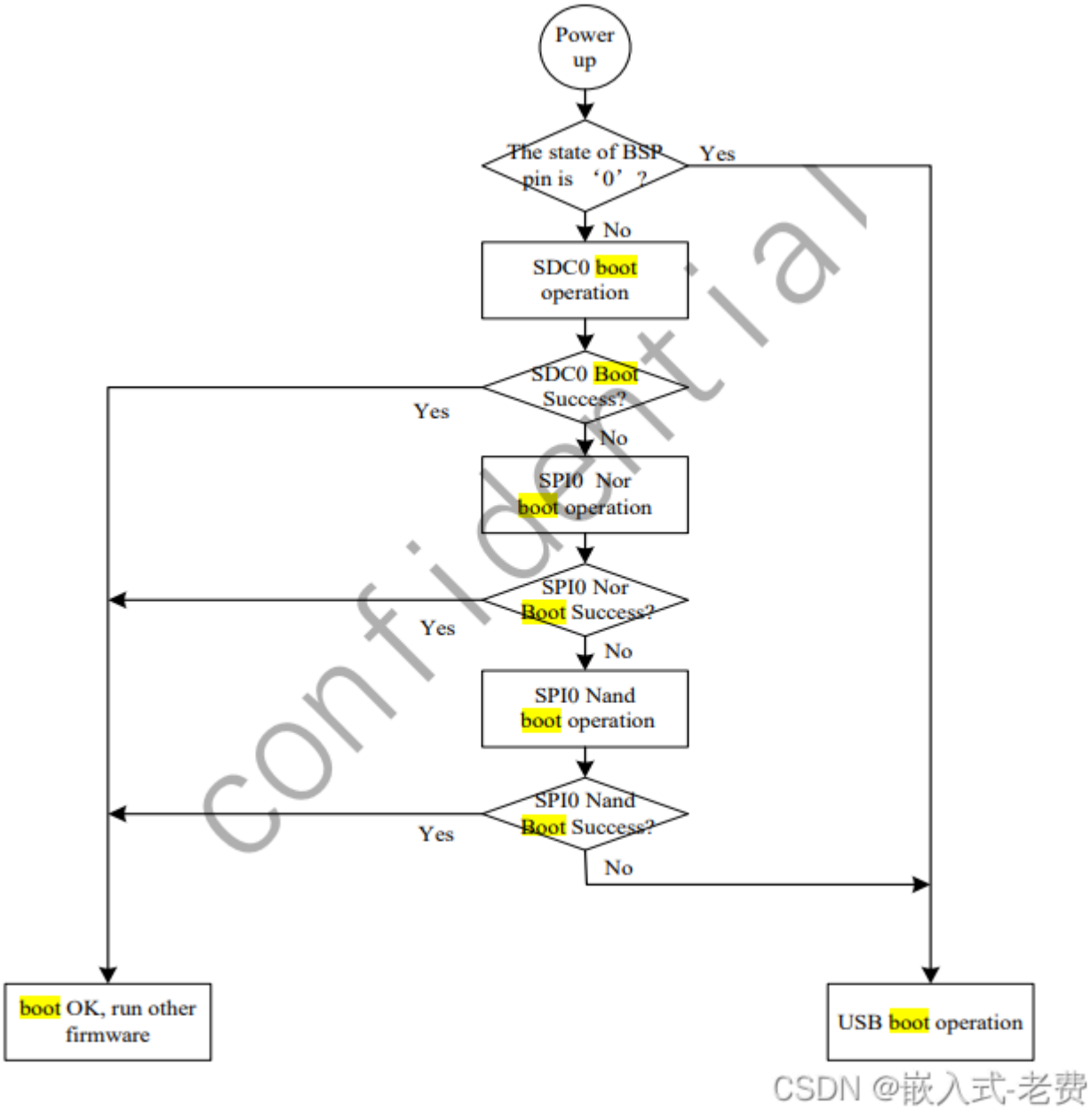
【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing@163.com】

nand flash相信大家并不陌生，现在很多的固态硬盘上面，其实有很多的nand flash。只不过根据存储单元，分成slc、mlc和tlc三种。早在差不多20年前，那个时候大家还都是学习s3c2440，标准的核心板就是soc+ddr+nandflash，或者是soc+ddr+norflash。那时，norflash一般是spi接口的，nandflash一般是专门的ip controller直接控制的。

后来随着nandflash成本的快速降低，出现了很多种nandflash的存储产品，这里面有nvme ssd、有sata ssd、有emmc、还有今天所说的spi nand flash。这些产品的共同特点就是，都使用了nandflash，但是外部的接口是不一样的，有的简单一些，有的比较复杂，甚至内部添加了很多的算法来解决nand flash固有的一些缺点。

之前我们在刚学v3s的时候，有一张v3s芯片手册上的启动图，不知道大家还有没有印象，

Open: 3808d9a1ea81e65ad723537945ef6d57_MD5.png



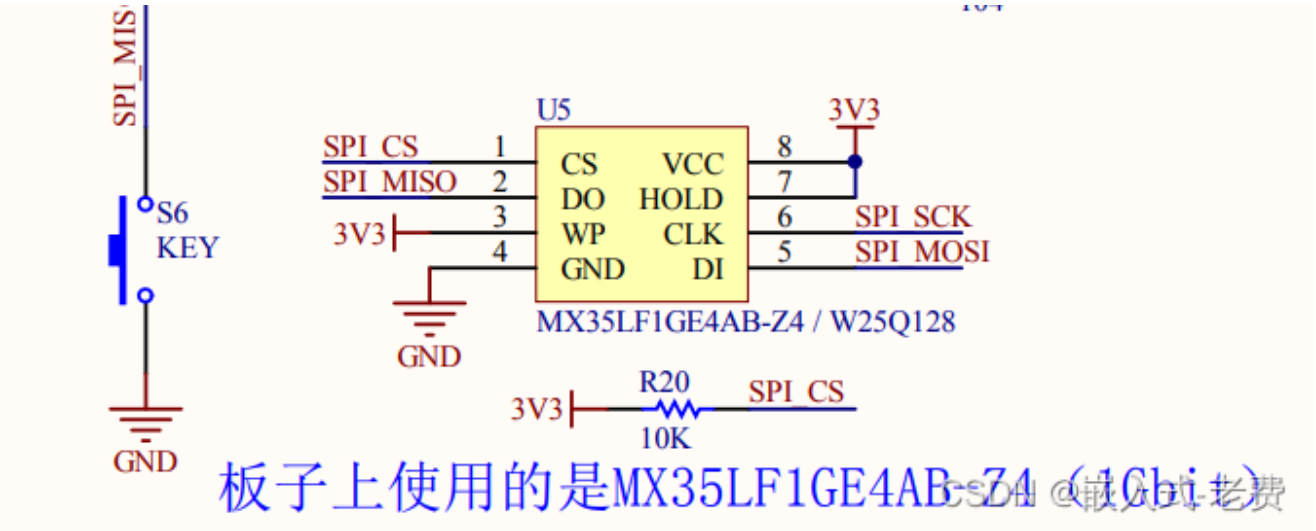
这张图清晰地说明了soc启动的四个顺序，分别是usb启动、sd卡启动、spi norflash启动、spi nandflash启动四种方式。前面我们测试的时候一直走的是sd卡启动，但是从成本和稳定性上说，spi nandflash其实是最合适的。

虽然spi norflash也可以当成v3s的启动媒介，但是一般来说，norflash不是很大，存储的东西有限。稍微多一点的图片、视频和动态库，norflash就放不下了。而sd卡虽然放的东西比较多，但是本身成本略贵，且由于是机械插入，在一些机器人领域，或者说存在震动场景的领域，sd卡也有点不太合适的。所以，综合来说，很多厂家都会选择spi nandflash当成启动和存储系统数据的基本介质，用户自己的sd卡可以作为补充，保存一些自己的资料，待v3s启动之后，mount到系统上面即可。

另外有一个比较有意思的现象，大家也可以注意下，那就是目前v3s和其他芯片沟通比较多的是哪几种协议。除了和flash沟通的spi，还有和点屏芯片沟通的gpio，和触控芯片沟通的iic，以及和wifi芯片沟通的sd接口。此外，串口232、485也经常用，不过这一类的驱动就不写在底层，一般是通过上层app去写，这一点和网络设备的驱动有点类似。

1、原理图

Open: b761ae4472bd417ebb6b1306592eb64a_MD5.png



系统使用的芯片是mx35lf1ge4ab，这部分大家不要看到spi接口的flash，就默认为norflash，其实可以把mx35lf1ge4ab的芯片手册找过来看下，上面就写的非常明白，

Open: fc40d3cc162495b38992a7395d05e8fa_MD5.png

MXIC
MACRONIX INTERNATIONAL CO., LTD.

MX35LF1GE4AB
MX35LF2GE4AB

2. GENERAL DESCRIPTIONS

The MX35LFxGE4AB is a 1Gb/2Gb SLC NAND Flash memory device with Serial interface.

The memory array of this device adopted the same cell architecture as the parallel NAND, however implementing the industry standard serial interface.

An internal 4-bit ECC logic is implemented in the chip, which is enabled by default. The internal ECC can be disabled or enabled again by command. When the internal 4-bit ECC logic is disabled, the host side needs to handle the 4-bit ECC by host micro controller.

CSDN @嵌入式-老费

接口部分就是常规的spi四线接口，这部分没有疑问，主要就是cs、clk、miso、mosi。这其中miso上有一个按键s6，这个需要注意下，主要用于usb烧入的时候使用，防止系统默认用spi nandflash加载系统。如果按键被按下，那么miso的数据就没有办法被v3s访问到，就不会从spi加载系统了。

2、设备树文件

默认sun8i-v3s-licheepi-zero.dtsi文件当中是没有spi的相关内容的。所以，这部分的脚本需要自己手动添加一下，

```
&spi0 {
        pinctrl-names = "default";
        pinctrl-0 = <&spi0_pins>;
        status = "okay";

        flash@0 {
                #address-cells = <1>;
                #size-cells = <1>;
                compatible = "spi-nand";
                reg = <0>;
                spi-max-frequency = <40000000>;

        partitions {
                compatible = "fixed-partitions";
                #address-cells = <1>;
                #size-cells = <1>;

            partition@0 {
                label = "all";
                reg = <0x000000 0x8000000>;
            };
        };
        };
};
```

3、驱动代码

关于驱动代码这部分，由于我们这次所要驱动的芯片是spi-nandflash的部分。所以驱动其实很容易就分成两部分，一部分是spi、一部分是nandflash。本质上来说，驱动就是用v3s的spi接口按照mx35lf1ge4ab芯片手册的命令要求，通过命令去访问或者写入flash里面的数据。一开始的时候，很自然地我们会去找drivers/spi和drivers/mtd/nand对应的驱动代码。

后来，随着一些资料的查找，我们发现在kernel 4.19之后，spi nandflash被作为独立的一个class，在驱动里面被单独分割了出来。从某种意义上说，只要升级kernel，修改一下.config配置文件，就可以实现相关的驱动加载了。

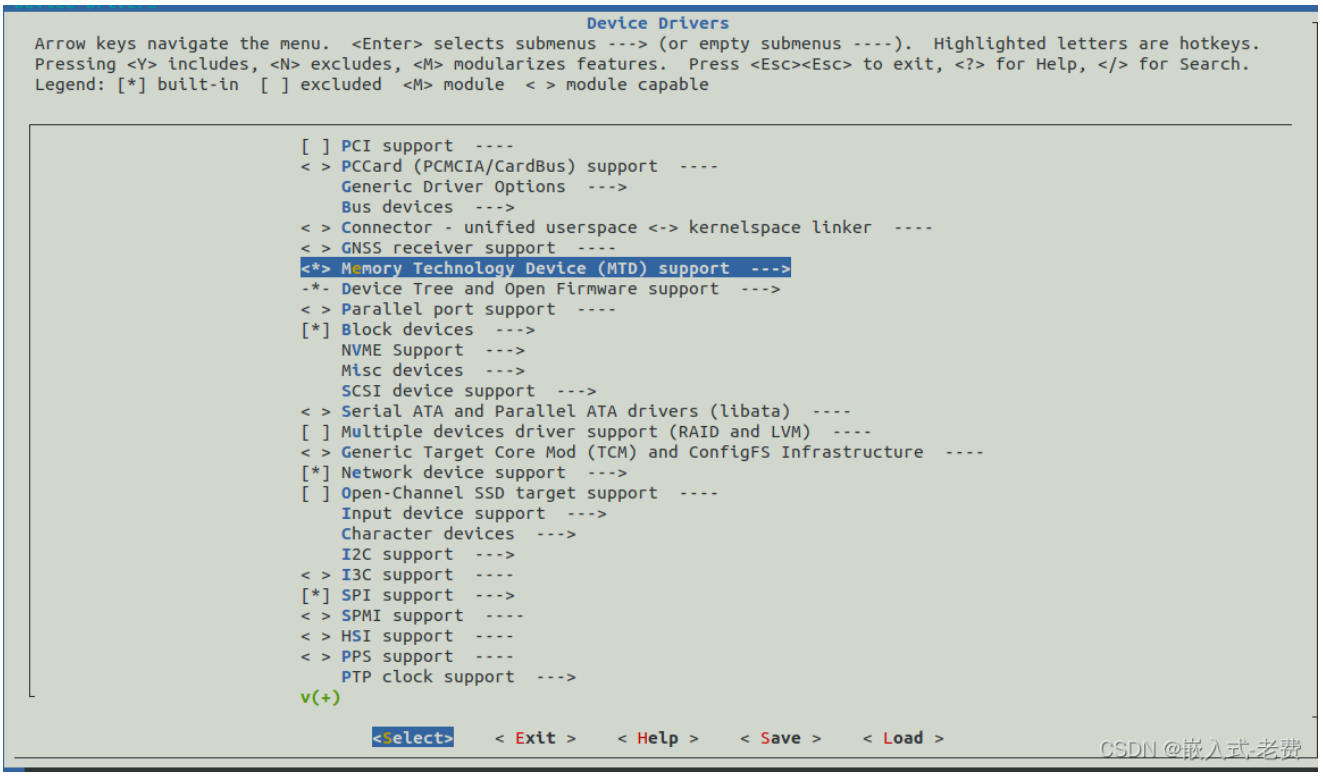
```
https://github.com/torvalds/linux/tree/v4.18/drivers/mtd/nand
https://github.com/torvalds/linux/tree/v4.19/drivers/mtd/nand
```

另外，结合v3s上面的linux分支代码，我们最终决定把内核升级到5.2.y，同时还需要进行两个设置。这里只是升级了kernel，编译器还是之前的6.3。首先，打开menuconfig配置表，

```
make ARCH=arm menuconfig
```

首先，使能mtd，

Open: f08f810254de7198deeff0da58549160_MD5.png



接着使能spi nand，

Open: c61e2ecb3c558de98eb6325489136484_MD5.png

```
                    Memory Technology Device (MTD) support
Arrow keys navigate the menu.  <Enter> selects submenus ---> (or empty submenus ----).  Highlighted letters are hotkeys.
Pressing <Y> includes, <N> excludes, <M> modularizes features.  Press <Esc><Esc> to exit, <?> for Help, </> for Search.
Legend: [*] built-in  [ ] excluded  <M> module  < > module capable

        --- Memory Technology Device (MTD) support
        < >   MTD tests support (DANGEROUS)
        < >   Command line partition table parsing
        <*>   OpenFirmware partitioning information support
        < >   TI AR7 partitioning support
              Partition parsers  --->
              *** User Modules And Translation Layers ***
        < >   Caching block device access to MTD devices
        < >     Readonly block device access to MTD devices
        < >   FTL (Flash Translation Layer) support
        < >   NFTL (NAND Flash Translation Layer) support
        < >   INFTL (Inverse NAND Flash Translation Layer) support
        < >   Resident Flash Disk (Flash Translation Layer) support
        < >   NAND SSFDC (SmartMedia) read only translation layer
        < >   SmartMedia/xD new translation layer
        < >   Log panic/oops to an MTD buffer
        < >   Swap on MTD device support
        [ ]   Retain master device when partitioned
              RAM/ROM/Flash chip drivers  --->
              Mapping drivers for chip access  --->
              Self-contained MTD device drivers  --->
        < >   OneNAND Device Support  ----
        < >   Raw/Parallel NAND Device Support  ----
        <*>   SPI NAND device Support  ----
              LPDDR & LPDDR2 PCM memory drivers  --->
        <*>   SPI-NOR device support  --->
        < >   Enable UBI - Unsorted block images  ----
        v(+)

              <Select>    < Exit >    < Help >    < Save >    < Load >
```

CSDN @嵌入式-老费

有了这两个部分，下面就可以开始编译了，

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-
```

有些同学害怕配置出错，担心把原来的配置文件弄坏，其实没有关系，这个时候只需要把之前的文件.config保存备份一下。如果后续实验成功了，只需要diff一下两个.config文件的区别是什么就可以了，

Open: e3294e355dc1fe96be8ced2e9867bb52_MD5.png

```
feixiaoxing@feixiaoxing-VirtualBox:~/Desktop/linux-zero-5.2.y$ diff .config .config_back
1012d1011
< CONFIG_MTD_NAND_CORE=y
1015c1014
< CONFIG_MTD_SPI_NAND=y
---
> # CONFIG_MTD_SPI_NAND is not set
```

CSDN @嵌入式-老费

4、烧入image和dtb文件

编译好之后，需要分别烧入image和dtb文件到sd卡，因为这两个文件都发生了改动。

```
sudo cp /home/feixiaoxing/Desktop/linux-zero-5.2.y/arch/arm/boot/zImage
.
sudo cp /home/feixiaoxing/Desktop/linux-zero-
5.2.y/arch/arm/boot/dts/sun8i-v3s-licheepi-zero-with-480x272-lcd.dtb .
```

5、开始测试和验证

查看mtd驱动是不是真的加载到开发板上面了，主要看两个地方就可以了。第一，就是看下启动起来的时候，日志里面有没有nandflash被正确加载的打印，

```
[    0.763876] spi-nand spi0.0: Macronix SPI NAND was found.
[    0.769366] spi-nand spi0.0: 128 MiB, block size: 128 KiB, page
size: 2048, OOB size: 64
[    0.778279] 1 fixed-partitions partitions found on MTD device spi0.0
[    0.784638] Creating 1 MTD partitions on "spi0.0":
[    0.789525] 0x000000000000-0x000008000000 : "all"
```

第二，等系统正常启动之后，可以ls -l /dev/mtd*一下，如果没有问题的话，应该可以看到对应的设备节点，

```
# ls -l /dev/mtd0*
crw-------    1 root     root       90,   0 Jan  1 00:00 /dev/mtd0
crw-------    1 root     root       90,   1 Jan  1 00:00 /dev/mtd0ro
```

有了上面的内容，基本就可以认为说，目前为止，nandflash已经被正确加载了。

6、后续的工作

目前的nandflash，只是作为sd卡启动后的一个外设存在的。我们对它的了解还比较基础，有兴趣的朋友可以移植一下mtd-utils工具，进一步对/dev/mtd0进行操作，相信会有更多的收获。